

Architectural Justification

Lab 3 — Component Modelling & Architectural Pattern Selection · OpenAPI Mock API Server

Name: Pranav Kalla SRN: PES1UG24AM197

We chose a **Microservices architecture** for the OpenAPI Mock API Server (the API mocking and testing platform described in the scenario).

Architectural Choice

The system is decomposed into small, independently deployable services — a Spec Manager, a Mock Generator, a Mock Request Handler, a Request Validator, a Response Engine and a Logging Service — each owning a single responsibility and communicating over well-defined interfaces. This was preferred over a Layered monolith (which couples all concerns into one deployable and scales as a single unit) and over a plain Client-Server design (which centralises logic and creates a single point of failure).

Two Scenario-Specific Reasons

1. **Independent scaling of the hot path.** The scenario requires sustaining at least 1,000 mock API requests per second while holding configured latency within tolerance. Only the Mock Request Handler and Response Engine sit on this high-throughput path; spec parsing and validation happen rarely, at design time. Microservices let us replicate just those runtime services horizontally without paying to scale the seldom-used Spec Manager, which a layered monolith cannot do.
2. **Fault isolation for untrusted input.** The system must parse externally supplied OpenAPI specifications and handle malformed or oversized files gracefully. Isolating parsing inside the Spec Manager means a crash or resource spike triggered by a hostile specification is contained to that service; the mock endpoints already serving QA engineers keep running, so validation work cannot take down live testing traffic.

Security Advantage

Because incoming traffic is funnelled through the Mock Request Handler and every request is checked by a dedicated Request Validator before any response is produced, schema validation and input sanitisation are enforced at a single, hardened boundary. Malformed requests are rejected with a 400 and undefined routes with a 404, and injection payloads never reach the Response Engine or the data store. Each service also runs with least-privilege access — only the Logging Service and Spec Manager touch persistence — so a compromise of a public-facing service does not grant direct database access.

Performance Benefit

Separating the Response Engine (which applies configurable latency, status codes and randomized schema-conformant payloads) from the Mock Request Handler lets the request path stay lightweight and stateless, so it can be load-balanced across many instances to reach the 1,000 req/s target. Simulated delays are applied asynchronously per request and logging is dispatched over an async message channel, meaning artificial latency and I/O for one request never block the throughput of others.